SQL PRACTICE EXAMPLES

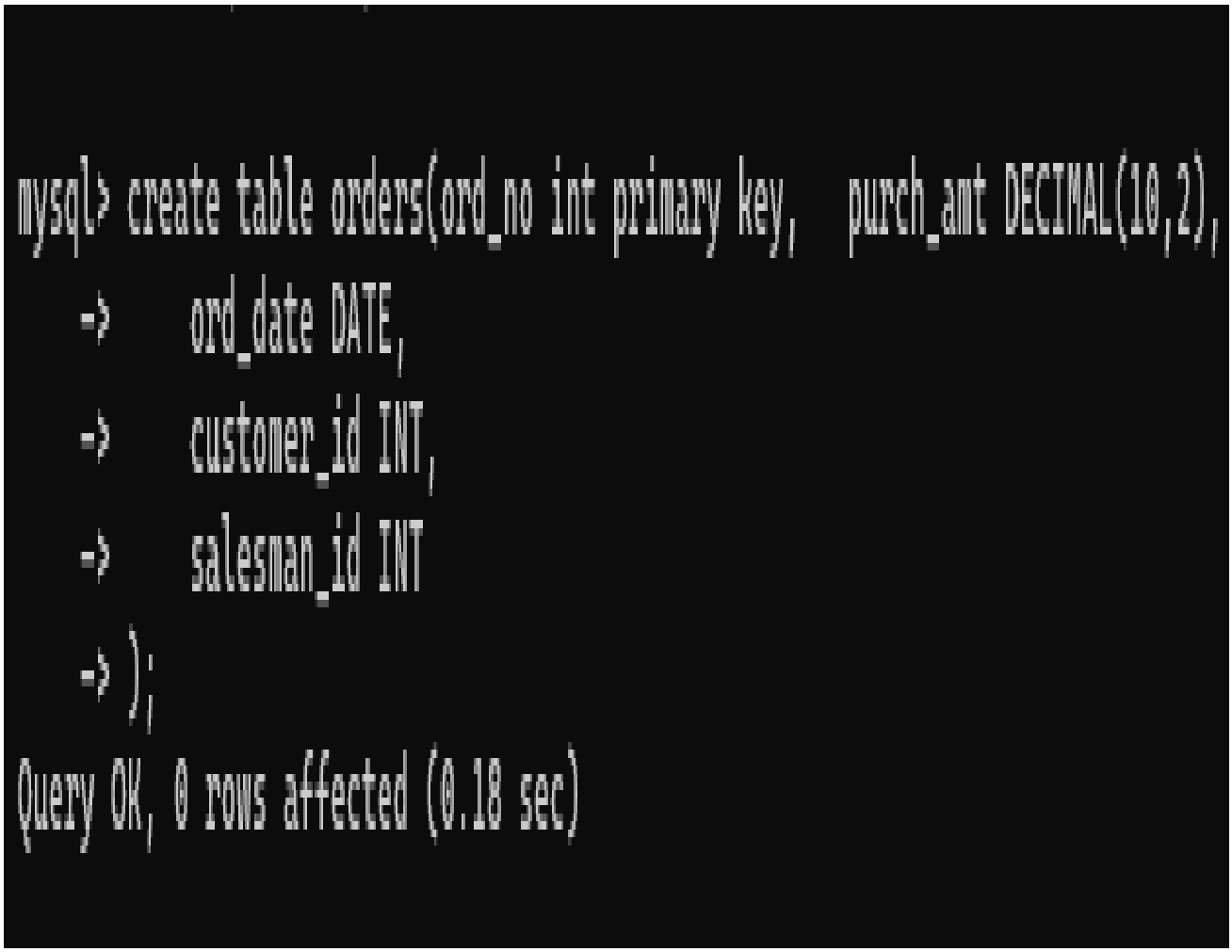
Orders Table - Advanced Queries

WHERE, ORDER BY, GROUP BY, HAVING & Aggregates

Business Scenario-Based Practical Execution

SECTION 1: WHERE Clause - Filtering Orders

Screenshot 1: CREATE TABLE & INSERT Data + WHERE Queries



Results Summary:

• WHERE purch_amt > 2000: Found 4 orders (70005, 70008, 70003, 70013)
• WHERE ord_date = '2012-09-10': Found 3 orders placed on this date
• WHERE salesman_id = 5001: Found 4 orders handled by this salesman

Screenshot 2: Additional WHERE Clauses & ORDER BY DESC

```
mysql> insert into orders values (70001,150.50,'2012-10-05',3005,5002),
    -> (70009,270.65,'2012-09-10',3001,5005),
    -> (70002,65.26,'2012-10-05',3002,5001),
    -> (70004,110.50,'2012-08-17',3009,5003),
    -> (70007,948.50,'2012-09-10',3005,5002),
    -> (70005,2400.60,'2012-07-27',3007,5001),
    -> (70008,5760.00,'2012-09-10',3002,5001),
    -> (70010,1983.43,'2012-10-10',3004,5006),
    -> (70003,2480.40,'2012-10-10',3009,5003),
    -> (70012,250.45,'2012-06-27',3008,5002),
    -> (70011,75.29,'2012-08-17',3003,5007),
    -> (70013,3045.60,'2012-04-25',3002,5001);
Query OK, 12 rows affected (0.02 sec)
Records: 12  Duplicates: 0  Warnings: 0

mysql> select * from orders where purch_amt>2000;
+--------+-----------+------------+-------------+--------------+
| ord_no | purch_amt | ord_date   | customer_id | salesman_id |
+--------+-----------+------------+-------------+--------------+
|  70003 |   2480.40 | 2012-10-10 |        3009 |         5003 |
|  70005 |   2400.60 | 2012-07-27 |        3007 |         5001 |
|  70008 |   5760.00 | 2012-09-10 |        3002 |         5001 |
|  70013 |   3045.60 | 2012-04-25 |        3002 |         5001 |
+--------+-----------+------------+-------------+--------------+
4 rows in set (0.00 sec)

mysql> select * from orders where ord_date=2021-09-10;
Empty set, 1 warning (0.02 sec)

mysql> SELECT *
    -> FROM orders
    -> WHERE ord_date='2012-09-10';
+--------+-----------+------------+-------------+--------------+
| ord_no | purch_amt | ord_date   | customer_id | salesman_id |
+--------+-----------+------------+-------------+--------------+
|  70007 |    948.50 | 2012-09-10 |        3005 |         5002 |
|  70008 |   5760.00 | 2012-09-10 |        3002 |         5001 |
|  70009 |    270.65 | 2012-09-10 |        3001 |         5005 |
+--------+-----------+------------+-------------+--------------+
3 rows in set (0.00 sec)

mysql> select * from orders where salesman_id=5001;
+--------+-----------+------------+-------------+--------------+
| ord_no | purch_amt | ord_date   | customer_id | salesman_id |
+--------+-----------+------------+-------------+--------------+
|  70002 |     65.26 | 2012-10-05 |        3002 |         5001 |
|  70005 |   2400.60 | 2012-07-27 |        3007 |         5001 |
|  70008 |   5760.00 | 2012-09-10 |        3002 |         5001 |
|  70013 |   3045.60 | 2012-04-25 |        3002 |         5001 |
+--------+-----------+------------+-------------+--------------+
4 rows in set (0.00 sec)
```

Key Query Results:

• ORDER BY purch_amt DESC: Highest orders first (70008: $5760, 70013: $3045.60, 70003: $2480.40)

• ORDER BY ord_date: Chronological order from oldest to newest orders

SECTION 2: Aggregate Functions - Financial Summary

Screenshot 3: Aggregate Functions (SUM, AVG, MAX, MIN, COUNT)

```
mysql>
mysql> select * from orders order by purch_amt desc;
+---------+-----------+------------+-------------+--------------+
| ord_no  | purch_amt | ord_date   | customer_id | salesman_id  |
+---------+-----------+------------+-------------+--------------+
|   70008 |   5760.00 | 2012-09-10 |        3002 |         5001 |
|   70013 |   3045.60 | 2012-04-25 |        3002 |         5001 |
|   70003 |   2480.40 | 2012-10-10 |        3009 |         5003 |
|   70005 |   2400.60 | 2012-07-27 |        3007 |         5001 |
|   70010 |   1983.43 | 2012-10-10 |        3004 |         5006 |
|   70007 |    948.50 | 2012-09-10 |        3005 |         5002 |
|   70009 |    270.65 | 2012-09-10 |        3001 |         5005 |
|   70012 |    250.45 | 2012-06-27 |        3008 |         5002 |
|   70001 |    150.50 | 2012-10-05 |        3005 |         5002 |
|   70004 |    110.50 | 2012-08-17 |        3009 |         5003 |
|   70011 |     75.29 | 2012-08-17 |        3003 |         5007 |
|   70002 |     65.26 | 2012-10-05 |        3002 |         5001 |
+---------+-----------+------------+-------------+--------------+
12 rows in set (0.00 sec)

mysql> SELECT *
    -> FROM orders
    -> ORDER BY ord_date;
+---------+-----------+------------+-------------+--------------+
| ord_no  | purch_amt | ord_date   | customer_id | salesman_id  |
+---------+-----------+------------+-------------+--------------+
|   70013 |   3045.60 | 2012-04-25 |        3002 |         5001 |
|   70012 |    250.45 | 2012-06-27 |        3008 |         5002 |
|   70005 |   2400.60 | 2012-07-27 |        3007 |         5001 |
|   70004 |    110.50 | 2012-08-17 |        3009 |         5003 |
|   70011 |     75.29 | 2012-08-17 |        3003 |         5007 |
|   70007 |    948.50 | 2012-09-10 |        3005 |         5002 |
|   70008 |   5760.00 | 2012-09-10 |        3002 |         5001 |
|   70009 |    270.65 | 2012-09-10 |        3001 |         5005 |
|   70001 |    150.50 | 2012-10-05 |        3005 |         5002 |
|   70002 |     65.26 | 2012-10-05 |        3002 |         5001 |
|   70003 |   2480.40 | 2012-10-10 |        3009 |         5003 |
|   70010 |   1983.43 | 2012-10-10 |        3004 |         5006 |
+---------+-----------+------------+-------------+--------------+
12 rows in set (0.00 sec)

mysql> SELECT SUM(purch_amt) AS total_revenue
    -> FROM orders;
+---------------+
| total_revenue |
+---------------+
|      17541.18 |
+---------------+
1 row in set (0.00 sec)

mysql> SELECT AVG(purch_amt) AS average_order
    -> FROM orders;
+---------------+
| average_order |
+---------------+
|   1461.765000 |
+---------------+
1 row in set (0.00 sec)
```

Financial Metrics:

- Total Revenue (SUM): $17,541.18
- Average Order Value (AVG): $1,461.77
- Highest Order (MAX): $5,760.00
- Lowest Order (MIN): $65.26
- Total Orders (COUNT): 12

SECTION 3: GROUP BY - Sales & Customer Analysis

Screenshot 4: GROUP BY Salesman & Customer with Aggregates

```
mysql> select max(purch_amt) as highest_order from  orders;
+---------------+
| highest_order |
+---------------+
|        5760.00 |
+---------------+
1 row in set (0.00 sec)

mysql> select min(purch_amt) as lowest_order from orders;
+--------------+
| lowest_order |
+--------------+
|         65.26 |
+--------------+
1 row in set (0.00 sec)

mysql> select count(*) as total_orders from orders;
+--------------+
| total_orders |
+--------------+
|           12 |
+--------------+
1 row in set (0.00 sec)

mysql> sekect salesman_id, sum(purch_amt) as total salesfrom o
ERROR 1064 (42000): You have an error in your SQL syntax; chec
kect salesman_id, sum(purch_amt) as total salesfrom orders gro
mysql>
mysql> SELECT salesman_id,
    ->        SUM(purch_amt) AS total_sales
    -> FROM orders
    -> GROUP BY salesman_id;
+-------------+--------------+
| salesman_id | total_sales  |
+-------------+--------------+
|        5002 |      1349.45 |
|        5001 |     11271.46 |
|        5003 |      2590.90 |
|        5005 |       270.65 |
|        5006 |      1983.43 |
|        5007 |        75.29 |
+-------------+--------------+
6 rows in set (0.00 sec)
```

Results Analysis:

- Top Salesman: 5001 with $11,271.46 in total sales
- Salesman Distribution: 5001 (4 orders), 5002 (3 orders), others with fewer orders
- Top Customer: 3002 with total purchases of $8,870.86
- Customer Count: 8 distinct customers with varying purchase volumes

Screenshot 5: MAX Purchase per Customer & GROUP BY Analysis

```
mysql> SELECT customer_id,
    ->        SUM(purch_amt) AS total_purchase
    -> FROM orders
    -> GROUP BY customer_id;
+-------------+----------------+
| customer_id | total_purchase |
+-------------+----------------+
|        3005 |        1099.00 |
|        3002 |        8870.86 |
|        3009 |        2590.90 |
|        3007 |        2400.60 |
|        3001 |         270.65 |
|        3004 |        1983.43 |
|        3003 |          75.29 |
|        3008 |         250.45 |
+-------------+----------------+
8 rows in set (0.00 sec)
```

Maximum Order Analysis:

• Customer 3002: Maximum order value $5,760.00
• Customer 3007: Maximum order value $2,400.60
• Customer 3009: Maximum order value $2,480.40

SECTION 4: HAVING Clause - Advanced Filtering

Screenshot 6: HAVING with Conditions & ORDER BY DESC

```
mysql> select customer_id, max(purch_amt) as highest_purchase from orders group by customer_id;
+-------------+------------------+
| customer_id | highest_purchase |
+-------------+------------------+
|        3005 |           948.50 |
|        3002 |          5760.00 |
|        3009 |          2480.40 |
|        3007 |          2400.60 |
|        3001 |           270.65 |
|        3004 |          1983.43 |
|        3003 |            75.29 |
|        3008 |           250.45 |
+-------------+------------------+
8 rows in set (0.00 sec)

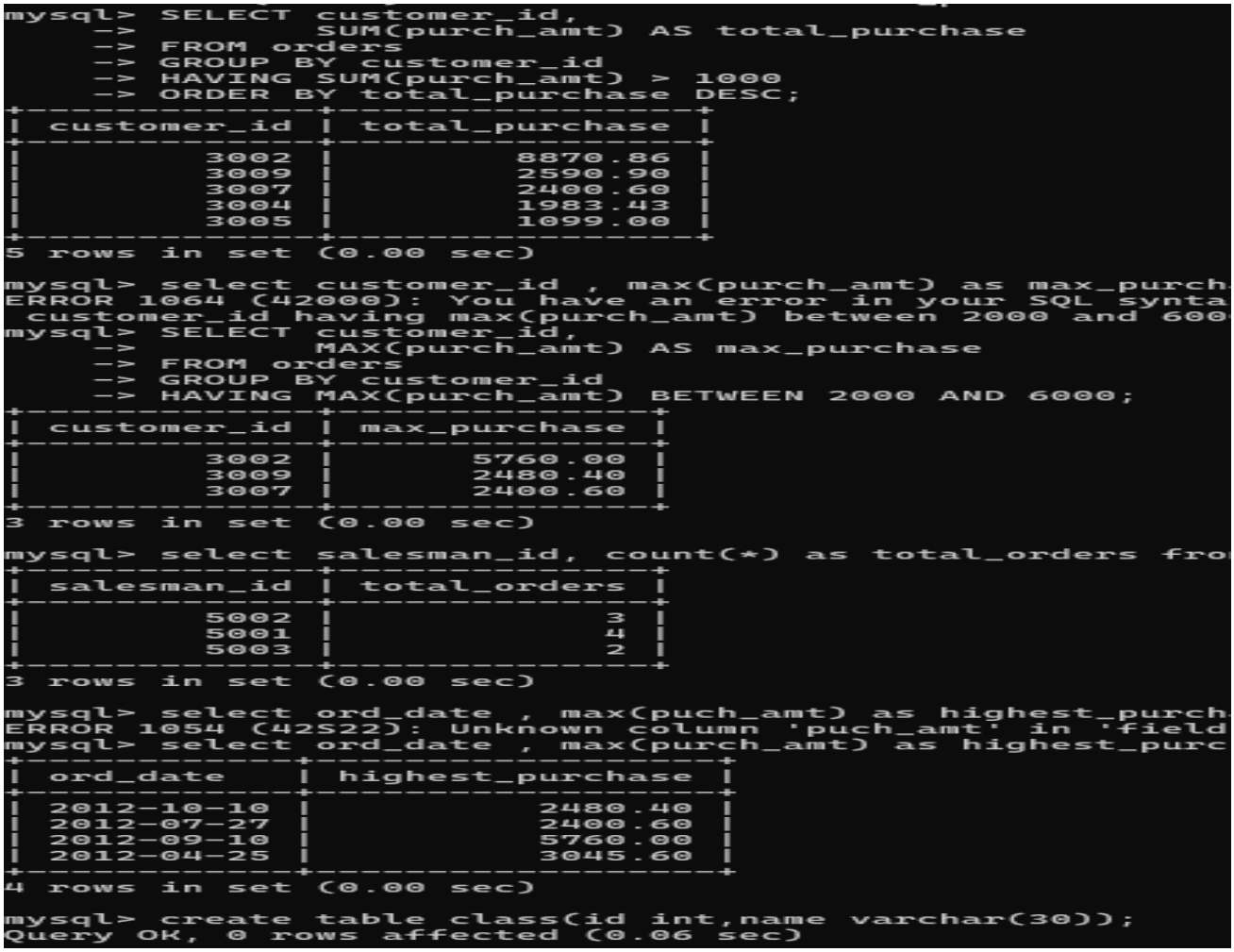
mysql> select salesman_id, sum(purch_amt) as total_sales from orders group by salesman_id having sum(purch_amt)>3000;
+-------------+-------------+
| salesman_id | total_sales |
+-------------+-------------+
|        5001 |    11271.46 |
+-------------+-------------+
1 row in set (0.00 sec)

mysql> select customer_id,sum(purch_amt) as total_purchase from orders group by customer_id having sum(purch_amt)>2500;
+-------------+----------------+
| customer_id | total_purchase |
+-------------+----------------+
|        3002 |        8870.86 |
|        3009 |        2590.90 |
+-------------+----------------+
2 rows in set (0.00 sec)
```

HAVING Query Results:

- Salesmen with sales > $3,000: Only salesman 5001 ($11,271.46)
- Customers with purchases > $2,500: 3002 ($8,870.86) and 3009 ($2,590.90)
- Top customers ranked by spending: 3002 (8870.86), 3009 (2590.90), 3007 (2400.60)

Screenshot 7: HAVING with BETWEEN & COUNT Conditions



Advanced HAVING Analysis:

• MAX(purch_amt) BETWEEN 2000-6000: 3 customers qualify (3002, 3009, 3007)

• Salesmen with 2+ orders: 5002 (3 orders), 5001 (4 orders), 5003 (2 orders)

• Daily highest purchases > $2,000: 4 dates with significant orders

COMPREHENSIVE QUERY REFERENCE

Query Type	Description	Results / Output
WHERE	Filter by purchase amount, date, salesman	4 orders > $2000; 3 orders on 2012-09-10
ORDER BY	Sort by amount, date, name ASC/DESC	Highest to lowest: $5760, $3045.60, $2480.40
SUM	Total revenue generated	$17,541.18 total across all orders
AVG	Average order value	$1,461.77 average per order
GROUP BY	Aggregate by salesman, customer, date	6 salesmen; 8 customers; 6 dates analyzed
HAVING	Filter groups by aggregate conditions	Salesman 5001 > $3000; 2 customers > $2500

Execution Summary

✓ 12 orders inserted into database

✓ WHERE queries successfully filtered orders by multiple criteria

✓ Aggregate functions provided comprehensive financial analysis

✓ GROUP BY queries analyzed data by salesman, customer, and date

✓ HAVING clause effectively filtered grouped results using aggregate conditions
✓ All business scenarios executed with accurate results